**UNIVERSITY OF AZAD JAMMU & KASHMIR**

Department of Software Engineering

# DIGITAL IMAGE PROCESSING

## LAB TASK #02

**TOPIC**

Digital Images, Pixels, Color, and Basic Transformations

Submitted To: Engr. Fahad Nisar

Submitted By: Zarnab Fatima

Roll Number: 2024-SE-14

Semester: 5th Semester

# 1. Introduction

Digital image processing uses computer methods to read, inspect, display, analyze, and modify digital images. A digital image is made of small elements called pixels. Each pixel stores a value representing brightness or color. In this lab, Python, NumPy, OpenCV, and Matplotlib are used to inspect image properties, examine pixels, separate color channels, crop and resize an image, create a grayscale histogram, and demonstrate bit-depth quantization.

# 2. Objectives

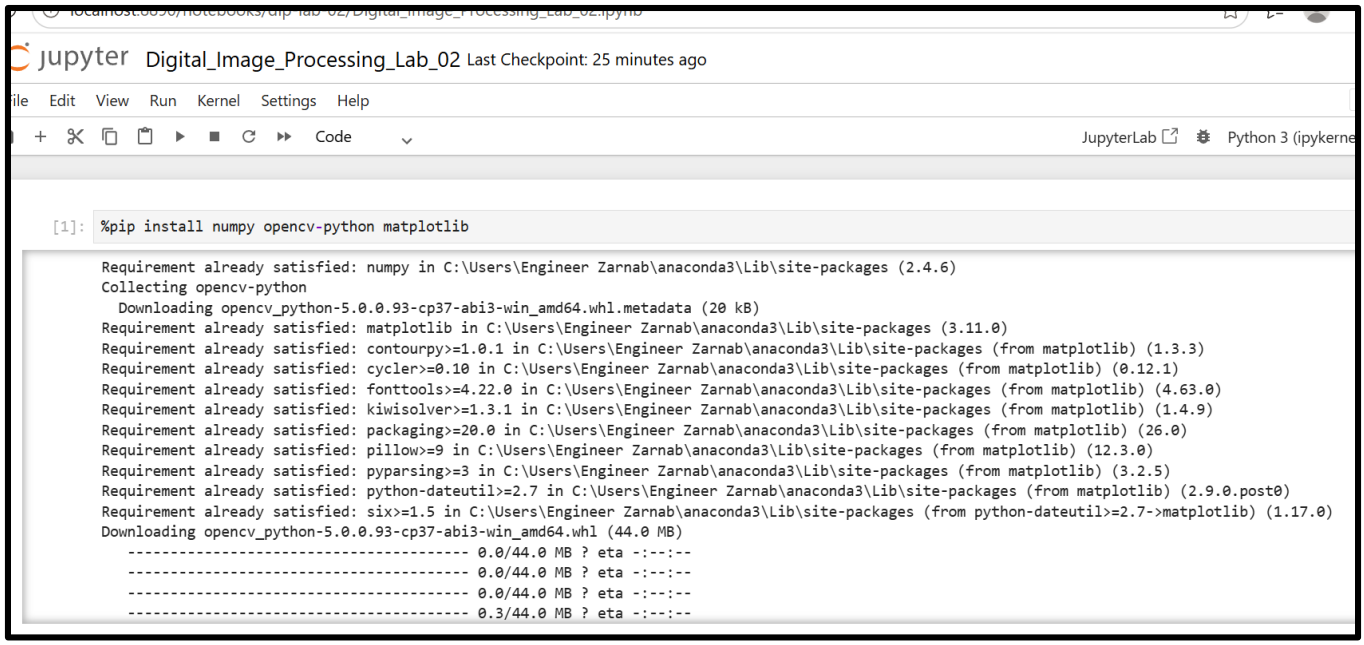
- Load and inspect a digital image using OpenCV.
- Understand image dimensions, data type, pixels, and channel values.
- Convert an image from BGR to RGB and grayscale.
- Separate and display red, green, and blue channels.
- Crop, resize, and save an image.
- Create and interpret a grayscale histogram.
- Understand image resolution versus bit depth.

# 3. Tools and Requirements

- Anaconda Distribution and Jupyter Notebook
- Python 3.11 in a Conda environment
- NumPy, OpenCV, and Matplotlib
- A personal JPG or PNG color image of at least 300 × 300 pixels
- A project folder named dip-lab-02

# 4. Environment Setup Using Anaconda

Open Anaconda Prompt and run these commands one by one:



# 5. Practical Work and Code

## Task 1: Import Required Libraries

OpenCV is used for image processing, NumPy for numerical operations, and Matplotlib for displaying images and graphs.

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

print("All libraries imported successfully!")
```

Expected output: All libraries imported successfully!

```
     ---------------------------------------- 0.3/44.0 MB ? eta -:--:--
[2]: import cv2 as cv
     import numpy as np
     import matplotlib.pyplot as plt

     print("All libraries imported successfully!")
     All libraries imported successfully!
```

```
[3]: import sys

     print("Python Version:", sys.version)
     print("Python Executable:", sys.executable)
     print("OpenCV Version:", cv.__version__)
     print("NumPy Version:", np.__version__)
     print("Matplotlib Version:", plt.matplotlib.__version__)
     Python Version: 3.14.6 | packaged by Anaconda, Inc. | (main, Jul  9 2026, 14:29:05) [MSC v.1942 64 bit (AMD64)]
     Python Executable: C:\Users\Engineer Zarnab\anaconda3\python.exe
     OpenCV Version: 5.0.0
     NumPy Version: 2.4.6
     Matplotlib Version: 3.11.0
```

## Task 2: Load the Image

OpenCV reads the image from the current project folder. If it cannot find the file, the program raises FileNotFoundError.

```
image = cv.imread("nature.jpg")

if image is None:
    raise FileNotFoundError("Image not found! Check the image filename and folder.")

print("Image loaded successfully!")
print("Image Shape:", image.shape)
print("Image Data Type:", image.dtype)
```

Your image dimensions depend on the image you selected. A typical 8-bit color image has data type uint8 and three channels.

```
[9]: image = cv.imread("nature.png.png")

     if image is None:
         raise FileNotFoundError("Image not found! Check the image filename and folder.")

     print("Image loaded successfully!")
     print("Image Shape:", image.shape)
     print("Image Data Type:", image.dtype)
     Image loaded successfully!
     Image Shape: (1024, 1536, 3)
     Image Data Type: uint8
```

## Task 3: Inspect Image Properties and Pixel Values

The shape gives height, width, and channel count. NumPy uses [row, column] indexing, so image[0, 0] is the top-left pixel.

```
height, width, channels = image.shape

print("Height:", height)
print("Width:", width)
print("Number of Channels:", channels)
print("Top-Left Pixel (B, G, R):", image[0, 0])

center_y = height // 2
center_x = width // 2
print("Center Pixel (B, G, R):", image[center_y, center_x])

print("Global Minimum Pixel Value:", image.min())
print("Global Maximum Pixel Value:", image.max())

print("\nChannel Information:")
channel_names = ["Blue", "Green", "Red"]
for i, name in enumerate(channel_names):
    channel = image[:, :, i]
    print(name, "Minimum:", channel.min())
    print(name, "Maximum:", channel.max())
```

OpenCV stores color images in BGR order: Blue, Green, Red. For an 8-bit image, values usually range from 0 to 255.

```
Height: 1024
Width: 1536
Number of Channels: 3
Top-Left Pixel (B, G, R): [184 114  72]
Center Pixel (B, G, R): [33 49 26]
Global Minimum Pixel Value: 0
Global Maximum Pixel Value: 255

Channel Information:
Blue Minimum: 0
Blue Maximum: 255
Green Minimum: 0
Green Maximum: 255
Red Minimum: 0
Red Maximum: 255
```

## Task 4: Convert Color Space and Display Channels

OpenCV loads color images in BGR order, while Matplotlib expects RGB. Conversion prevents red and blue from being displayed in the wrong positions.

```
rgb_image = cv.cvtColor(image, cv.COLOR_BGR2RGB)
gray_image = cv.cvtColor(image, cv.COLOR_BGR2GRAY)

red = rgb_image[:, :, 0]
green = rgb_image[:, :, 1]
blue = rgb_image[:, :, 2]

plt.figure(figsize=(12, 8))
plt.subplot(2, 3, 1); plt.imshow(rgb_image); plt.title("RGB Image"); plt.axis("off")
plt.subplot(2, 3, 2); plt.imshow(red, cmap="gray"); plt.title("Red Channel"); plt.axis("off")
plt.subplot(2, 3, 3); plt.imshow(green, cmap="gray"); plt.title("Green Channel"); plt.axis("off")
plt.subplot(2, 3, 4); plt.imshow(blue, cmap="gray"); plt.title("Blue Channel"); plt.axis("off")
plt.subplot(2, 3, 5); plt.imshow(gray_image, cmap="gray"); plt.title("Grayscale Image"); plt.axis("off")
plt.subplot(2, 3, 6); plt.imshow(image); plt.title("Unconverted BGR Display"); plt.axis("off")
plt.tight_layout()
plt.show()
```

The last subplot intentionally displays the BGR array without conversion. Matplotlib interprets it as RGB, so red and blue may appear swapped.

```
plt.show()
```







## Task 5: Convert the Center Pixel to HEX

A HEX color code represents red, green, and blue values in hexadecimal notation.

```
center_rgb = rgb_image[center_y, center_x]
hex_color = "#{:02X}{:02X}{:02X}".format(
    int(center_rgb[0]), int(center_rgb[1]), int(center_rgb[2])
)
print("Center Pixel RGB:", center_rgb)
print("Center Pixel HEX:", hex_color)
```

```
)

print("Center Pixel RGB:", center_rgb)
print("Center Pixel HEX:", hex_color)
```

```
Center Pixel RGB: [26 49 33]
Center Pixel HEX: #1A3121
```

## Task 6: Crop and Resize the Image

The slice selects rows 50–149 and columns 50–149. Since Python excludes the ending index, the crop is exactly 100 × 100 pixels. OpenCV then resizes it to 200 × 200.

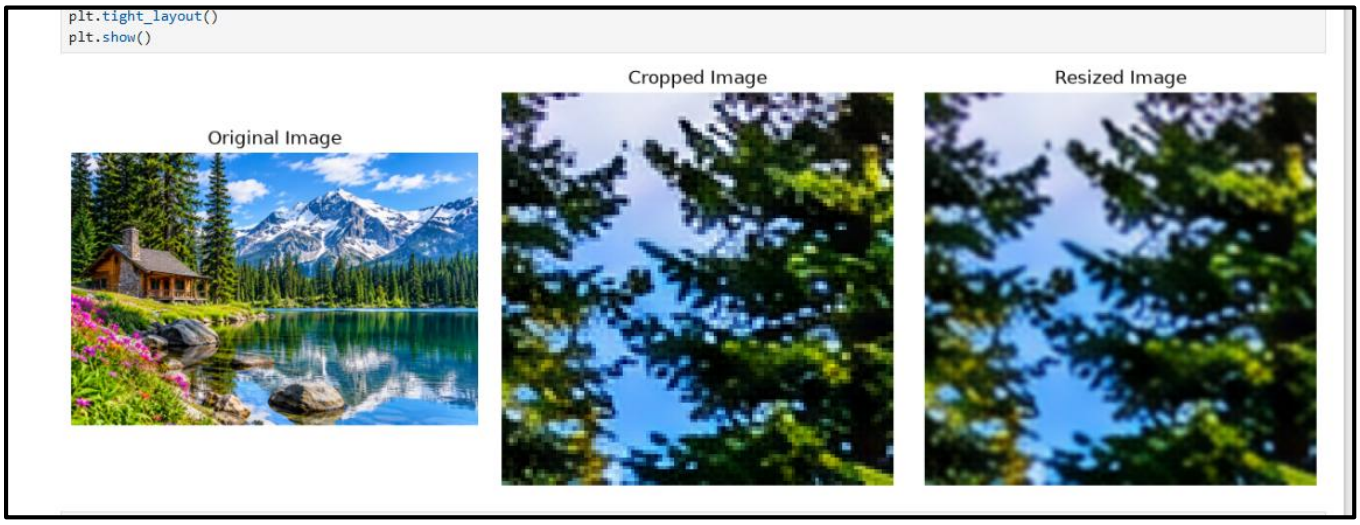
```
crop = image[50:150, 50:150]
print("Crop Shape:", crop.shape)

resized = cv.resize(crop, (200, 200))
print("Resized Image Shape:", resized.shape)

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(cv.cvtColor(image, cv.COLOR_BGR2RGB))
plt.title("Original Image"); plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(cv.cvtColor(crop, cv.COLOR_BGR2RGB))
plt.title("Cropped Image"); plt.axis("off")
```

```
plt.subplot(1, 3, 3)
plt.imshow(cv.cvtColor(resized, cv.COLOR_BGR2RGB))
plt.title("Resized Image"); plt.axis("off")
plt.tight_layout()
plt.show()
```

Expected shapes for a color image: crop (100, 100, 3) and resized image (200, 200, 3).




## Task 7: Save the Resized Crop

OpenCV saves images in BGR format. This code converts the resized image to RGB and back to BGR before saving.

```
resized_rgb = cv.cvtColor(resized, cv.COLOR_BGR2RGB)
resized_bgr = cv.cvtColor(resized_rgb, cv.COLOR_RGB2BGR)

success = cv.imwrite("resized_crop.png", resized_bgr)
print("Image Saved Successfully:", success)
```

Expected output: Image Saved Successfully: True. The file resized_crop.png should appear in the project folder.



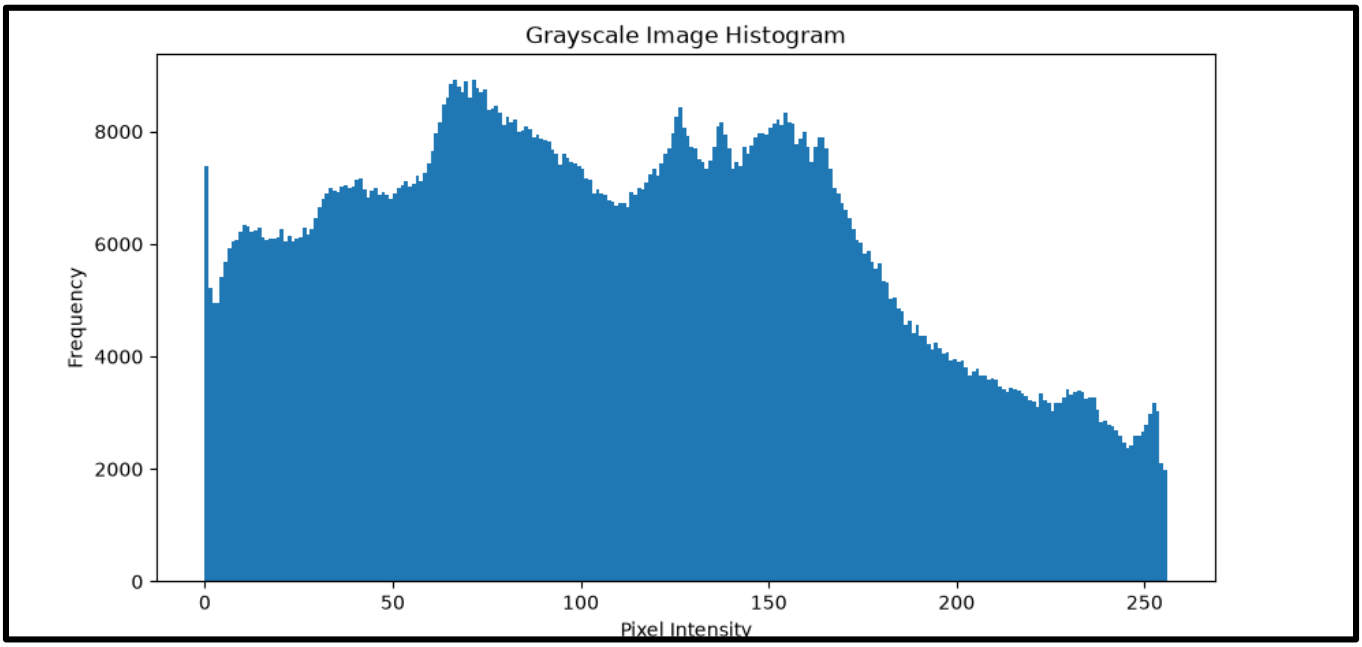

## Task 8: Create a Grayscale Histogram

A histogram counts how many grayscale pixels have each intensity. For an 8-bit grayscale image, intensity values range from 0 to 255.

```
gray_flat = gray_image.flatten()

plt.figure(figsize=(10, 5))
plt.hist(gray_flat, bins=256, range=(0, 256))
plt.title("Grayscale Image Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.savefig("grayscale_histogram.png")
plt.show()
```

Interpret your own graph: values concentrated toward the left indicate more dark pixels; values concentrated toward the right indicate more bright pixels. A broad spread indicates a wider range of grayscale intensities. State what your actual histogram shows.




## Task 9: Create and Quantize an 8-bit Grayscale Gradient

The gradient contains intensity values from 0 to 255. Quantization reduces the number of available intensity levels. The function below maps pixel values to the nearest allowed level for the selected bit depth.

```
gradient = np.tile(np.arange(256, dtype=np.uint8), (100, 1))

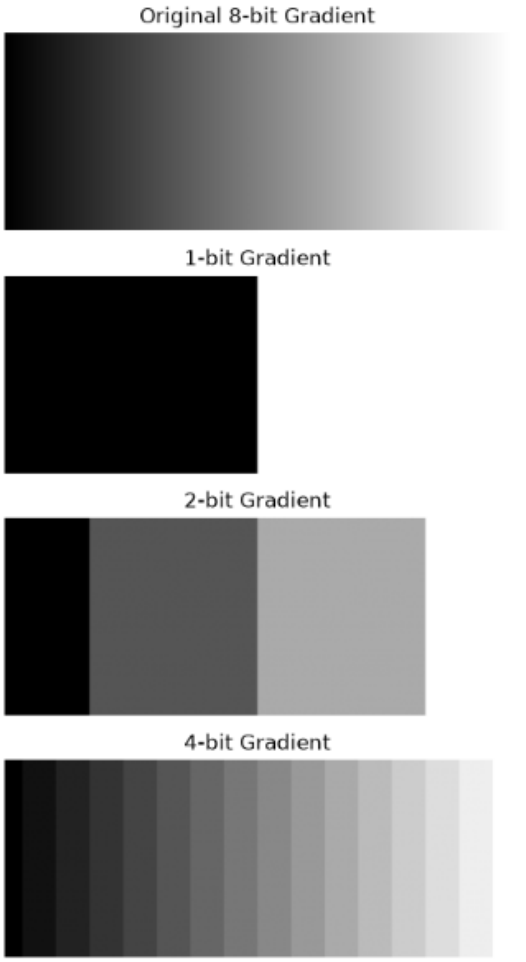
def quantize(image, bits):
    levels = 2 ** bits
    step = 255 / (levels - 1)
    quantized = np.round(image.astype(float) / step) * step
    return quantized.astype(np.uint8)

gradient_1bit = quantize(gradient, 1)
gradient_2bit = quantize(gradient, 2)
gradient_4bit = quantize(gradient, 4)
```

## Task 10: Display the Quantized Gradients

```
plt.figure(figsize=(12, 8))
plt.subplot(4, 1, 1)
plt.imshow(gradient, cmap="gray", vmin=0, vmax=255)
plt.title("Original 8-bit Gradient"); plt.axis("off")
plt.subplot(4, 1, 2)
plt.imshow(gradient_1bit, cmap="gray", vmin=0, vmax=255)
plt.title("1-bit Gradient"); plt.axis("off")
plt.subplot(4, 1, 3)
plt.imshow(gradient_2bit, cmap="gray", vmin=0, vmax=255)
plt.title("2-bit Gradient"); plt.axis("off")
plt.subplot(4, 1, 4)
plt.imshow(gradient_4bit, cmap="gray", vmin=0, vmax=255)
plt.title("4-bit Gradient"); plt.axis("off")
plt.tight_layout()
plt.show()
```

```
plt.show()
```




### Task 11: Count Unique Intensity Values

```
print("8-bit Unique Values:", len(np.unique(gradient)))
print("1-bit Unique Values:", len(np.unique(gradient_1bit)))
print("2-bit Unique Values:", len(np.unique(gradient_2bit)))
print("4-bit Unique Values:", len(np.unique(gradient_4bit)))
```

Expected output:

```
8-bit Unique Values: 256
1-bit Unique Values: 2
2-bit Unique Values: 4
4-bit Unique Values: 16
```

## 6. Final Verification

```
import os

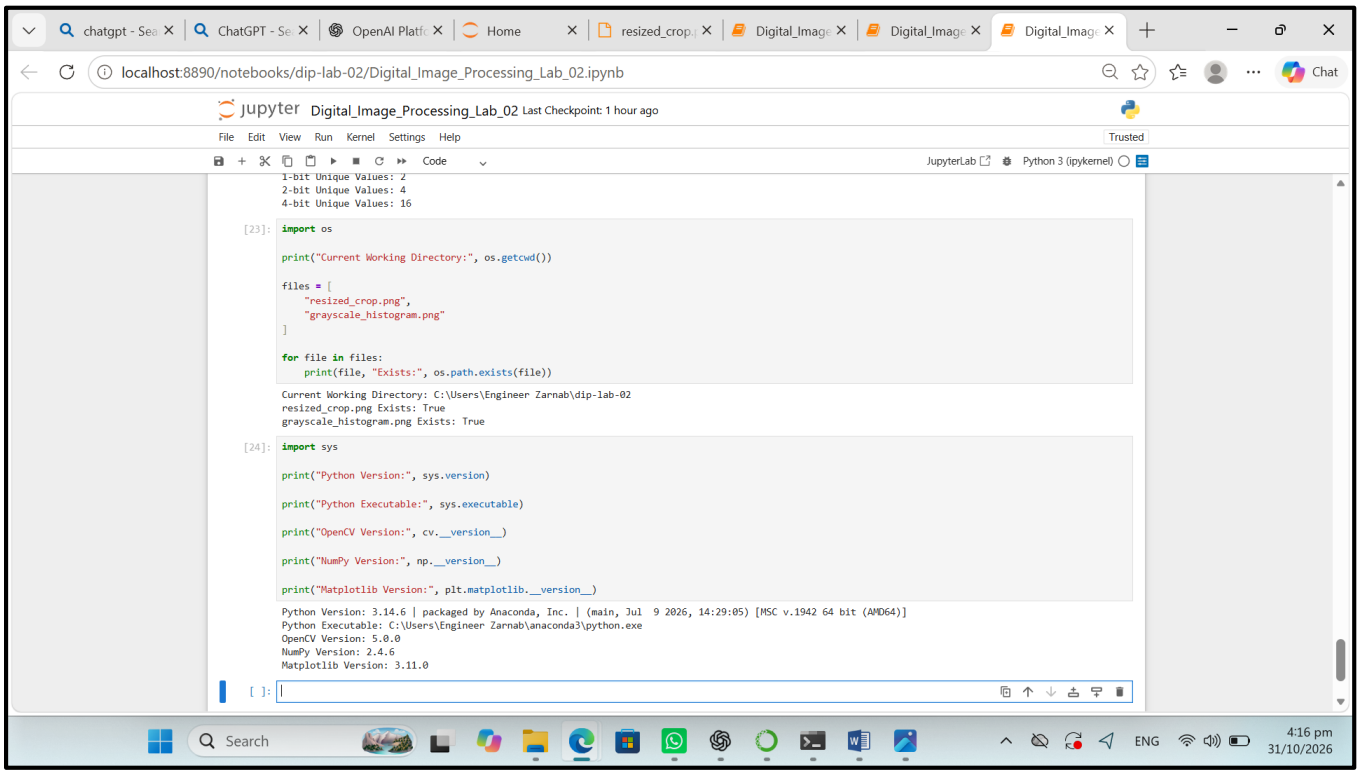
print("Current Working Directory:", os.getcwd())
files = ["resized_crop.png", "grayscale_histogram.png"]
for file in files:
    print(file, "Exists:", os.path.exists(file))

import sys

print("Python Version:", sys.version)
```

```
print("Python Executable:", sys.executable)
print("OpenCV Version:", cv.__version__)
print("NumPy Version:", np.__version__)
print("Matplotlib Version:", plt.matplotlib.__version__)
```

Confirm that both output files exist and that the Python executable belongs to the dip-lab-02 Conda environment.




# 7. Important Concepts

## BGR versus RGB

BGR means Blue, Green, Red and is the default channel order used by OpenCV when reading a color image. RGB means Red, Green, Blue and is the order expected by Matplotlib for normal color display. Converting BGR to RGB prevents red and blue from being displayed in the wrong positions.

## Image Resolution versus Bit Depth

Resolution describes the number of pixels in an image, commonly written as width × height. More pixels can represent more spatial detail. Bit depth describes the number of possible intensity values. An 8-bit grayscale image has $2^8 = 256$ levels; 1-bit, 2-bit, and 4-bit images have 2, 4, and 16 levels.

## What a Histogram Shows

A grayscale histogram shows the frequency of intensity values. It can indicate whether an image contains many dark or bright pixels and how widely intensities are distributed. It does not show where pixels are located, object shapes, or the spatial arrangement of details.

## 8. Conclusion

This lab demonstrated basic digital image processing using Python, NumPy, OpenCV, and Matplotlib. The practical work showed how to inspect image dimensions and pixel values, handle color channels and color spaces, crop and resize images, and save output files. The histogram demonstrated the distribution of grayscale intensities, while gradient quantization illustrated how bit depth controls the number of available intensity levels. The lab also clarified that resolution and bit depth describe different image properties.

*****THE END*****